Chapter 5 · Lesson 1 — When to Split a Component

Chapter 5 · Lesson 1 — When to Split a Component

> *Goal: develop the senior judgement of when JSX deserves its own component file. Two extremes are wrong; the middle is craft.*

The two failure modes

Beginners fail in opposite ways:

Failure mode 1 — the giant component. Everything in one file. 800 lines. One `return` statement that wraps a hero, a nav, three tabs, a footer, all inline. Hard to read, hard to test, hard to find anything.

Failure mode 2 — over-extraction. Every `<div>` is its own component. `<TeamNameLabel>`, `<TeamNameLabelInner>`, `<TeamNameLabelInnerSpan>`. Hard to read for the opposite reason — every variable name is a click-through to another file.

Senior engineers live in the middle. **Split when splitting buys you something.** Don't split for splitting's sake.

The three real reasons to split

Every legitimate component extraction passes one of these tests:

Test 1: The copy-paste test

If you're about to copy a chunk of JSX with minor variations, **extract first**.

Example from our codebase, inside `<MatchPickAnalysis>`:

```
<div>
  <div>BACKED {playerName.toUpperCase()}</div>
  {teamsBackedThis.map(t => <TeamChip team={t} ptsEarned={ptsForPicker} />)}
</div>
<div>
  <div>BACKED {opponent.toUpperCase()}</div>
  {teamsBackedOpponent.map(t => <TeamChip team={t} ptsEarned={ptsForOpponent
</div>
```

The two `<TeamChip>` map calls are **identical structure**, only the input list and the points value differ. If `<TeamChip>` were inlined as 20 lines of JSX, copy-pasting it twice would smell. Extracted, both call sites are one line.

The rule: **if you're about to copy-paste 10+ lines of JSX, extract first.**

Test 2: The "what vs how" test

If a chunk of JSX is conceptually about *one specific thing* and the rest of the component is about *something else*, the chunk should be its own component.

The parent says **what** (`<PathStep label="QF" status="won" />`). The child knows **how** (which colors, which icon, which sizes for each status).

Example: the tournament path indicator at the top of `<PlayerDetail>`:

```
<PathStep label="R1" status={r1Won ? 'won' : 'lost'} />
<PathArrow />
<PathStep label="R2" status={r2Won ? 'won' : 'lost'} />
<PathArrow />
<PathStep label="QF" status={qfFinished ? (qfWon ? 'won' : 'lost') : 'live'}
<PathArrow />
<PathStep label="SF" status={...} />
<PathArrow />
<PathStep label="F"  status={...} />
```

Imagine inlining this. Each `<PathStep>` is ~20 lines of JSX (status colors, icon, padding, border). Multiplied by 5 = 100 lines. Plus 4 inline `<PathArrow>` spans. The parent component would lose all sense of its **own** structure under the weight of the path-rendering code.

Extracted, the parent reads like a sentence: *"step, arrow, step, arrow, step."* That clarity is the win. **The component name is documentation.**

Test 3: The reusability test

If a chunk of JSX will be used in 2+ places (now or soon), it deserves its own component.

`<StatCard>` from Chapter 3 is used 3 times in the standings (Leader, Matches, Avg Score). It's used again in `<PlayerDetail>` for per-player stats. **Four uses. Obvious extraction.**

But: don't pre-extract for *hypothetical* reuse. Wait until you have the second use case. Premature extraction often picks the wrong abstraction; you end up retro-fitting the component to the second use case and making it worse for both.

> **Rule of three.** Some authors say: extract on the third use. The first time you write something, write it. The second time, copy-paste and feel a small pang of guilt. The third time, extract. By then you understand the abstraction well enough to design it right.

For our codebase, we extract on the **second** use because the codebase is small and TypeScript catches misuse. For larger projects, three is fine.

The case for tiny components

`<PathArrow>` is **one character**:

```
export default function PathArrow() {
  return (
    <span style={{ color: 'rgba(255,255,255,0.45)', fontSize: 18, fontWeight
  );
}
```

It has no props. It does no logic. It's just a styled →. Why is it a component?

Because **the parent's JSX reads like a sentence with it**:

```
<PathStep label="R1" status="won" />
<PathArrow />
<PathStep label="R2" status="won" />
```

```
<PathArrow />
<PathStep label="QF" status="lost" />
```

Inlined, the same code:

```
<PathStep label="R1" status="won" />
<span style={{ color: 'rgba(255,255,255,0.45)', fontSize: 18, fontWeight: 70
<PathStep label="R2" status="won" />
<span style={{ color: 'rgba(255,255,255,0.45)', fontSize: 18, fontWeight: 70
<PathStep label="QF" status="lost" />
```

The reader's eye loses the structure under the styled spans. The component **names the visual unit** and lets the structure breathe.

Tiny components like this aren't about reuse or DRY — they're about **readability of the parent**. The parent component is the customer; the tiny component serves it.

When NOT to split

Three signals that an extraction is wrong:

Signal 1: The component takes too many props.

```
// Smell — 8 props is too many
<TeamRow
  rank={i + 1}
  name={team.name}
  icon={team.icon}
  total={team.scores.total}
  r1={team.scores.r1}
  r2={team.scores.r2}
  finalPick={team.final}
  isLeader={i === 0}
/>
```

Eight props mean the abstraction isn't right. Either:

- The component should just accept the whole `team` object: `<TeamRow team={team}`

`rank={i+1} />`. That hides the data shape behind one prop.

- The component is doing too many jobs and should be split into smaller ones.

In our codebase, components rarely have more than 4–5 props. **5 props is the warning line.** 8+ is "rethink this."

Signal 2: The component has logic that depends on its parent's context.

```
function TeamRow({ team, parentTabState, parentSelectedTeam, parentSetters }
  // ...
}
```

If a child component accepts a bag of "parent state" as a prop, it's leaking the parent's internals into the child. Either lift the **logic** to the parent (and pass primitives down), or restructure so the child genuinely only needs primitives.

Signal 3: Extraction makes the parent harder to read.

```
<TeamSection>
  <TeamSectionHeader>
    <TeamSectionHeaderTitle>{title}</TeamSectionHeaderTitle>
  </TeamSectionHeader>
  <TeamSectionBody>...</TeamSectionBody>
</TeamSection>
```

Five layers of components for what could be a single `<section>` with two `<div>`s. **Don't introduce abstractions you can't justify.**

If extraction makes the parent component longer and the new file barely justifies its own existence, undo the extraction.

The composition pattern: parent says what, child knows how

This is the central pattern of React composition:

- **Parent components** are about **layout and orchestration**. They render lists, switch between views, decide which children show up. Their JSX is structural.

- **Child components** are about **rendering one thing well**. They take props, return JSX, don't know about the parent.

The classic test: can you delete the child component file and copy its JSX back inline, without touching the parent's logic? If yes, the split is clean. If you have to drag parent state down with you, the split was leaky.

Walkthrough: extracting `<TeamChip>`

Let's see this in action. Inside `<MatchPickAnalysis>` you have 8 team chips per match — 16 if you count both sides. The chip looks like:

```
<div style={{
  background: '#FFFFFF',
  borderRadius: 8,
  padding: '8px 12px',
  display: 'flex',
  alignItems: 'center',
  gap: 10,
  border: `2px solid ${team.accent}66`,
  boxShadow: `0 2px 6px ${team.accent}20`,
}}>
  <div style={{ fontSize: 22 }}>{team.icon}</div>
  <div style={{ flex: 1, fontWeight: 700, color: '#1F2937', fontSize: 14 }}>
  {ptsEarned !== null && (
    <div style={{
      background: ptsEarned === 3 ? '#16A34A' : '#92400E',
      color: '#FFFFFF',
      padding: '3px 10px',
      borderRadius: 6,
      fontSize: 12,
      fontWeight: 800,
    }}>
      +{ptsEarned}
    </div>
  )}
</div>
```

15 lines. Used 16+ times per match. Extract:

```
// components/players/TeamChip.tsx
import type { Team } from '@/lib/types';

interface TeamChipProps {
  team: Team;
  ptsEarned: number | null;
}

export default function TeamChip({ team, ptsEarned }: TeamChipProps) {
  return (
    <div style={{ /* 5 styles */ }}>
      <div>{team.icon}</div>
      <div>{team.name}</div>
      {ptsEarned !== null && (
        <div style={{ background: ptsEarned === 3 ? '#16A34A' : '#92400E', /*
          +{ptsEarned}
        </div>
      )}
    </div>
  );
}
```

The parent's call site is now one line:

```
{teamsBackedThis.map(t => <TeamChip key={t.name} team={t} ptsEarned={ptsForP
```

Cleaner. Reusable. The parent is shorter. The chip has a name. **All three benefits at once — that's a good extraction.**

Walkthrough: when NOT to extract `<TwoColumnLayout>`

Inside `<MatchPickAnalysis>`, the layout is:

```
<div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: 16 }}>
  <div>...left column with chips...</div>
  <div>...right column with chips...</div>
```

```
</div>
```

Could we extract `<TwoColumnLayout>`?

```
<TwoColumnLayout>
  <div>...left column...</div>
  <div>...right column...</div>
</TwoColumnLayout>
```

Maybe. But:

- It's used **once** in this codebase. **Rule of three** says wait.
- The grid template is 4 lines of CSS. Saving 4 lines isn't worth a new file.
- The name is generic — `TwoColumnLayout` is at the wrong level of abstraction (CSS), not domain (snooker fantasy).

So we **don't** extract. The CSS stays inline. **Discipline of saying no.**

The `children` prop — the most underused React feature

When you DO extract a layout component, you use the `children` prop:

```
function Card({ children, accent }: { children: React.ReactNode; accent: str
  return (
    <div style={{ borderLeft: `4px solid ${accent}`, padding: 16, background
      {children}
    </div>
  );
}

// Usage:
<Card accent="#0F5132">
  <h3>Round 1 Picks</h3>
  <PicksList details={...} />
</Card>
```

The `children` prop receives whatever JSX is between the opening and closing tags. Type it as `React.ReactNode` (the most permissive React type — accepts elements, strings, numbers, arrays, null, undefined, fragments).

Why this matters: `children` lets you build **layout components** that don't care what's inside them. The parent describes the contents; the layout component just wraps them.

In our codebase, `<ChartCard>` (which we'll see in Chapter 6) uses `children` exactly this way:

```
<ChartCard title="Points Breakdown" subtitle="...">
  <ResponsiveContainer width="100%" height={360}>
    <BarChart>...</BarChart>
  </ResponsiveContainer>
</ChartCard>
```

The card knows about title and subtitle. The chart inside is whatever the parent passes. **Layout-vs-content separation in five lines.**

A note on file structure

In this codebase we organize by **feature folder**:

```
components/
├── standings/   ← things only used in standings
├── matches/     ← things only used in matches
├── predictions/ ← things only used in predictions
├── players/     ← things only used in players
├── analytics/   ← things only used in analytics
└── tabs/        ← the top-level tab containers
```

Alternative would be by **type** (`/components/charts`, `/components/cards`, `/components/buttons`). For small apps, by-type works. For anything with more than ~20 components, **by-feature** scales better — you can find everything related to "the standings page" by opening one folder.

The `tabs/` folder is the exception; it's where the top-level tab containers live, since they don't fit cleanly into one feature folder. They're more like "feature entry points."

This is interview gold too: *"How do you organize React components?"* Answer: *"By feature, not by type. Each folder owns the leaf components for that feature, plus a top-level tab/page container."*

Vibe prompt you would have used

> *"This `<MatchPickAnalysis>` component is getting long. Extract two pieces. (1) The colored pill showing a team's icon + name + points-earned chip — call it `<TeamChip>`, props `{ team: Team, ptsEarned: number | null }`. (2) The small status step in the tournament-path bar — call it `<PathStep>`, props `{ label: string, status: 'won' | 'lost' | 'live' | 'na' }`, each mapping to colors and icon (□ □ □ —). Also extract the literal → arrow into a one-line `<PathArrow>` so the path JSX reads like 'step arrow step arrow step'.* ***Don't change any visuals — just move the JSX into the new files and import them back.*** *"*

That last sentence is the magic. Without it, the LLM "improves" the styling while moving the JSX, and the visual diff between before and after is bigger than expected. **"Don't change visuals — just move JSX"** is what turns a refactor back into a refactor.

CHECK YOURSELF

1. **`<PathArrow>` is a single styled span. What's the argument FOR keeping it as a component instead of inlining?**
2. **Why doesn't `<MatchPickAnalysis>` extract the two-column layout into its own component?** What three things would have to be true to justify it?
3. **A teammate proposes `<TeamSection>` containing `<TeamSectionHeader>` containing `<TeamSectionTitle>`. What questions do you ask?** What would the smell be?
4. **You see a 5-prop component: `<Foo a b c d e />`. Is that automatically too many?** When is it fine?
5. **Open `components/players/PlayerDetail.tsx`. List every component it imports.** For each, classify: copy-paste rule, what-vs-how rule, reusability rule, or readability rule.

When you've answered these, head to **02-prop-drilling-vs-context.md**.